Scanner Architecture and Evaluation Report

Vulnerability-Class Remediation Harness — scan/find phase

Field	Value
Generated	2026-07-27 02:04 UTC
Repository	Cybersecurity-Coding-Agent-Harness
Scope of this document	Stages 0 through 4 of the scanner (find phase). The patch/verify phase is future work and is not covered.
Score source of truth	results/eval-history/*.jsonl (append-only). This PDF is generated from those files by tools/eval/generate_eval_report.py and is not hand-edited.

1. Pipeline architecture

Six stages. Recon establishes what the target is; the lane selector decides what to hunt and where; the budget governor bounds every lane before any spend; hunt lanes do the reasoning in parallel; validation independently re-checks each candidate; output assembles the final artifact.

Stage	Name	Determinism	Build status
0	Recon	AST/filesystem extraction (deterministic) + 2 LLM passes	Built
0.5	Dynamic Lane Selector	Deterministic instantiation + 1 LLM verification pass	Built
1	Budget Governor	Fully deterministic (arithmetic, no model call)	Built
2	Hunt Lanes	N parallel LLM lanes, 3 phases	Built
3	Validate	1 LLM pass per consolidated finding	Built, redesign pending
4	Output	Schema assembly, no model call expected	Not started

2. Per-stage input / output contracts

Stage 0 — Recon

Direction	Contract
Input	Target repository root. Path is supplied via `--target` CLI argument or `SCANNER_TARGET` environment variable. No answer-key or category material is ever provided.
Output 1	`architecture-summary.json` — route table (method / path / handler / middleware / auth / file / line), auto-CRUD resource table with per-model exclude lists, persistence-layer map, dependency list, swagger coverage diff, client-side escape-hatch findings, LLM tool-calling verdict, file inventory, framework detection.
Output 2	`category-applicability.json` — one row per OWASP Top 10 / API Security Top 10 / LLM Top 10 category: verdict (present / absent / uncertain), evidence string, confidence 0-1.
Consumed by	Stage 0.5 (lane instantiation) and Stage 1 (budget sizing needs the seed footprint).

Stage 0.5 — Dynamic Lane Selector

Direction	Contract
Input	Stage 0's architecture summary + category applicability table.
Output	`lane-manifest.json` — array of lanes, each with `lane_id`, `categories[]`, `subsystem_scope`, `seed_files[]`, `playbook_reference`.
Special rules	(a) A permanent seed-file denylist excludes 3 benchmark-infrastructure files from every lane. (b) Unclassified-surface files from the inventory are filtered (non-executable languages dropped) then bin-packed into size-capped shards. (c) Detected smart-contract files get a dedicated lane.

Direction	Contract
Consumed by	Stage 1 (needs lane count + seed footprint) and Stage 2 (lane assignments).

Stage 1 — Budget Governor

Direction	Contract
Input	Lane manifest + on-disk byte size of every seed file.
Output	`budget-plan.json` — per lane: `model_tier`, `token_ceiling`, `wall_clock_ceiling`, `escalation_flag` plus the reason the flag was set.
Method	Arithmetic only, no model call. Ceilings are sized from each lane's own seed footprint; escalation is flagged when a lane's footprint sits in the top quartile by bytes or by file count.
Design intent	Per-lane ceilings, deliberately not one global cap, so a single expensive lane cannot starve the others.

Stage 2 — Hunt Lanes

Direction	Contract
Input	Architecture summary + lane assignment + category playbook + seed file contents (line-numbered) + budget ceiling + optional Semgrep hints (additive context only, never a filter).
Output 1	`candidate-findings.json` — per finding: `finding_id`, `lane_id`, `categories[]`, `title`, `description`, `trace[]` of {kind, file, line, description} where kind is entrypoint / propagation / sink, `severity_estimate`, `confidence`.
Output 2	`budget-consumption.json` — per lane: `tokens_used`, `seconds_elapsed`, `ceiling_hit`.
Phases	Phase 1 hunt across all lanes; Phase 2 orchestrator review of scope requests and escalation candidates; Phase 3 approved second passes. Findings are deduplicated before emission.
Validity rules	A finding is dropped unless its trace is non-empty, starts with an entrypoint step, and ends with a sink step.

Stage 3 — Validate

Direction	Contract
Input	One consolidated finding's claim (trace + impact) only. The hunting lane's reasoning transcript and lane identity are deliberately withheld so the check is blind and adversarial.
Output	`validated-findings.json` — per entry: `consolidated_id`, `original_finding_ids[]`, `original_lane_ids[]`, `verdict`, `validator_evidence`, plus carried-through title, trace and severity. Budget consumption is tracked per consolidated finding.
Consolidation	Overlapping candidates are merged before dispatch using the same plus/minus 15-line slack the scorer uses, so consolidation and scoring agree on what counts as the same finding.
Pending redesign	Current verdicts are CONFIRMED / REJECTED. The agreed direction is to move this stage to an annotator model (confidence plus concerns, no reject power). Not yet implemented.

Stage 4 — Output

Direction	Contract
Input	Stage 3 verdicts plus validator evidence.
Output	Schema-validated `findings.json` plus a human-readable summary, and a thin SARIF projection for interop. Schema follows security-audit-skill's report schema with two additions: `categories[]` (multi-label rather than one forced category) and `lane_provenance`.
Status	Not started. Contract above is the design target, not a description of existing code.

3. Evaluation methodology

3.1 Universal record — captured for every run of every component

These fields are mandatory regardless of which stage is being measured. A score without them is not interpretable later.

Field	Why it is mandatory
run_id / timestamp	Identifies the run so history can be diffed.
component + version (git SHA)	Ties a score to an exact reproducible artifact.
ground_truth_set (name + size)	Scores are meaningless without knowing what they were measured against.
models_used	Cost and quality both hinge on model tier.
tokens (in / out / cached)	Primitive usage measure; survives pricing changes.
est_cost_usd	Derived from tokens and current pricing; recorded as null when pricing is not established.
wall_clock_time	Latency and total compute-time diverge under parallel lanes; both are recorded.
primary correctness metrics	Defined per component below.
delta_vs_baseline	The 'did it improve' signal, computed against both the prior run and the best-known run.
pass / fail vs target	Turns numbers into a decision rather than just data.

3.2 Scanner correctness metrics — exact definitions in force

Metric	Exact definition as implemented
File match	Suffix-based path normalization between a candidate trace step's file and the ground-truth file, so absolute vs repo-relative paths compare equal.
Recall	Ground-truth entries for which some candidate finding has a trace step that file-matches AND sits at the EXACT ground-truth line (slack = 0) AND whose category codes intersect the ground-truth entry's codes. Divided by total ground-truth entries.
Localization	Same as recall but with a line slack of plus/minus 15 lines instead of exact match. Reported as the softer 'how close did it get' diagnostic.
File-match rate	Ground-truth entries where any candidate touched the right file at any line, ignoring category. Diagnostic only: separates 'never looked at the file' from 'looked but mislocalized or miscategorized'.
Precision proxy	Candidate findings that localize to some real ground-truth entry within 15 lines, ignoring category, divided by total candidate findings. Named a proxy because unmatched findings are not automatically false positives — some are real bugs outside the fixed benchmark set.
Category codes	Extracted by regex over category labels: A01-A10, API1-API10, LLM01-LLM10. Ground-truth codes are pre-encoded in the answer key rather than re-extracted per run.

Deliberate asymmetry: recall is the STRICTER bar (exact line) and localization the softer one (15 lines). This is intentional and was set explicitly — a scanner that names the wrong line still hands a patcher the wrong place to edit.

Multi-vulnerability lines: the ground truth contains locations carrying genuinely distinct vulnerability classes at one identical line (verified: 4 such locations). Each is a separate ground-truth entry and must be found and categorized independently to count. Credit is not shared across categories at the same line.

3.3 Per-stage evaluation method

Stage	What is measured	Method
0 Recon	Category-applicability accuracy (diagnostic)	Compare present/absent/uncertain verdicts against the categories actually exercised by the ground truth. Isolates whether a recall miss originated in lane selection rather than hunting.
0 Recon	Surface-discovery completeness	Every source file in the target must be accounted for as analyzed, divided, or excluded-with-reason. Deterministic assertion, not a judgement call.

Stage	What is measured	Method
0.5 Lane Selector	Seed coverage	For each ground-truth entry, is its file present in at least one lane's seed set. Directly separates 'not seeded' misses from 'seeded but not found' misses.
1 Budget Governor	Ceiling correctness	Cross-check per-lane ceilings in budget-plan.json against actual usage in budget-consumption.json. Pass condition: no lane silently exceeds its ceiling, and ceiling_hit is reported truthfully.
2 Hunt Lanes	Recall / localization / precision proxy, overall and per lane	The definitions in 3.2, applied to candidate-findings.json. Per-lane breakdown attributes a miss to a specific playbook.
2 Hunt Lanes	Prompt-reachability (added after a defect was found here)	For each ground-truth line, confirm it was actually inside the prompt text sent to some lane. Distinguishes a reasoning failure from a truncation failure.
3 Validate	Validator precision / recall	Of the raw candidates, how many did validation correctly keep versus correctly kill. Isolates whether a false result originated in the hunter or the validator.
4 Output	Schema conformance	Structural validation against the findings schema. Not a correctness check.

3.4 Whole-scanner evaluation

Metric	Definition
Precision / Recall / F1	Aggregate across the full pipeline output, against the complete ground-truth set.
Total pipeline cost and time	Summed across all stages for one full run; parallel-lane wall clock reported separately from summed compute time.
Run-to-run stability	Variance in findings across repeated runs on identical input. Agentic components are stochastic. Measured only at milestone points, since repeating an eval N times multiplies its cost by N.
Out-of-scope findings	Real findings outside the fixed ground-truth set. Tracked, not scored — signals whether the scanner finds genuine bugs beyond the benchmark.
Delta vs previous version	Trend across harness iterations over calendar time, not only within one component.

Operative targets in force: precision at or above 95%, recall at or above 90%, localization at or above 90%. The 100/100/100 observed ceiling is a reference point, not a definition of done.

4. Datasets

Property	juice-shop-benchmark-v1	juice-shop-benchmark-v2
Ground-truth entries	10	98
Purpose	External five-tool comparison	Full-scope scanner evaluation
Derivation	10 hand-verified challenges spanning distinct OWASP categories	All 113 built-in challenges, minus 15 excluded: 12 judged non-vulnerabilities, 3 reclassified during verification and recorded with reasons
Confidence	Hand-verified	All 98 entries at high confidence after three verification passes
Schema per entry	challengeKey, category, file, line, solveCondition, referenceCorrectFix, exploitTest	challengeKey, category, owasp_codes, file, line, confidence, note (the original 10 retain the richer fields)
Target application	juice-shop-blind	juice-shop-blind
Target size	same as v2	918 source files inventoried
Language mix	same as v2	TypeScript 531, JSON 151, CSS 86, HTML 82, YAML 46, JavaScript 10, Solidity 5, Python 3, Docker 2, Shell 2
Attack surface	same as v2	148 hand-written routes, 28 auto-CRUD resources, 16 models

Both sets measure the same target application; v2 supersedes v1 for scanner evaluation. The answer key lives in a separate restricted repository and is never made available to any scanner-building or scanning agent.

Statistical caveat that must travel with any v1 number: n=10. A single miss swings recall by 10 points. All four completing tools hit 100% precision partly because the set is small enough that none happened to misfire on it. Directional only.

Contamination caveat that must travel with any number from either set: OWASP Juice Shop is heavily publicly documented. A model may recognize it from pretraining and reconstruct known bugs from memory rather than genuine analysis. Neither set proves generalization to an unfamiliar codebase; only a held-out obscure or synthetic target would.

5. Recorded evaluation results

5.1 External five-tool baseline — dataset juice-shop-benchmark-v1 (10 challenges)

Tool	Precision	Recall	Localization	Findings in-scope / total
security-audit-skill (Cloudflare)	100.0%	100.0% (10/10)	100.0%	15 / 23
deepsec (Vercel Labs)	100.0%	90.0% (9/10)	100.0%	22 / 101
VulnHunter (Capital One)	100.0%	70.0% (7/10)	100.0%	8 / 37
raptor (community)	100.0%	20.0% (2/10)	50.0%	2 / 8
VVAH (Visa)	n/a	n/a	n/a	0 / 0

security-audit-skill (Cloudflare) — Highest scorer. Parallel LLM-reasoning lanes by attack class plus a separate adversarial validator. This architecture is the direct basis for this harness's Stage 2 + Stage 3 design.

deepsec (Vercel Labs) — Deterministic prefilter + real LLM investigation + revalidation. Ran at a reduced thinking level and skipped its optional revalidate step under budget pressure, still scored 90% recall.

VulnHunter (Capital One) — Attacker-first forward analysis. Hit an org-wide Opus spend cap mid-run - an operational failure, not a reasoning failure. Direct motivation for this harness's per-lane (not global) budget ceilings in Stage 1.

raptor (community) — Pattern-matching only. Missed every logic-heavy class (access control, auth bypass, prompt injection). This is the empirical floor that justifies 'prefilter may hint, never gate' in Stage 2.

VVAH (Visa) — DID NOT COMPLETE. 11-stage exhaustive pipeline timed out before emitting any report. Its own estimator correctly predicted ~4M input tokens/pass across 9 stages before running, but the pipeline had no mechanism to act on that estimate and scope down. Direct motivation for Stage 1 being a hard-stop governor rather than an advisory estimator.

5.2 This harness's scanner — dataset juice-shop-benchmark-v2 (98 challenges)

Run	Version	Recall (exact line)	Localization (15 lines)	File match	Precision proxy	Findings
2026-07-25-a	df7d0d6	30.6% (30/98)	37.8% (37/98)	67.3% (66/98)	53.7% (22/41)	41
2026-07-25-b	1f2fa23	27.6% (27/98)	41.8% (41/98)	71.4% (70/98)	63.2% (24/38)	38

Run	Stage 2 tokens	Lane seconds (summed)	Lanes	Ceiling hits
2026-07-25-a	238,494	286.0	16	0
2026-07-25-b	247,038	250.1	16	0

scanner-2026-07-25-a (df7d0d6) — First full-scope scored run against the complete 98-entry ground truth. INVALIDATING DEFECT: the misconfiguration lane failed entirely on an upstream HTTP 400 content filter (0 tokens, 0 findings), removing all A05/API8/API9 coverage. Scope requests: 5 (3 approved, 2 denied). Escalation second-passes: 5 reviewed, 4 approved.

scanner-2026-07-25-b (1f2fa23) — Stage 2 re-run after the PEM-key sanitization fix. misconfiguration lane now completes (15,299 tokens, 3 findings). Ran against the PRE-sharding 16-lane manifest, so it does not include the unclassified-code shards. Recall moved DOWN 3.0 pts while localization moved UP 4.0, file-match UP 4.1, and precision proxy UP 9.5 - a mix of the recovered lane and genuine run-to-run stochastic variance. Treat the a-vs-b delta as noise-dominated, not as evidence the fix hurt recall.

Status against target: both recorded runs are far below the operative floor (recall at or above 90%, localization at or above 90%). FAIL. The defects in section 6 account for a substantial and quantified share of the gap; they are not an explanation for all of it.

6. Known defects qualifying the current scores

These are confirmed by direct inspection, not inferred. Any reading of section 5 that ignores them will misattribute infrastructure failures to reasoning quality.

#	Defect	Evidence	Ground-truth impact	Status
1	Seed files truncated at 15,000 characters before the prompt is built	server.ts is 40,967 numbered characters (2.7x the cap) and is seeded to all 16 lanes, so every lane reads only its first ~37%. Byte offset of line 477 is 22,805.	5 entries physically unreachable: changeProductChallenge, registerAdminChallenge, adminSectionChallenge, forgedFeedbackChallenge, errorHandlingChallenge	Open
2	Findings inherit their lane's category list rather than naming their own	hunt-executor.ts assigns f.categories = lane.categories. The hunt prompt's required-output list never asks the model for a category. All 14 producing lanes emitted exactly one distinct category list each.	Breaks scoring in both directions: correct findings miss on category, and blanket lists grant credit that may not be earned	Open
3	misconfiguration lane rejected by upstream content filter	HTTP 400 with 0 tokens consumed. Root cause: a literal PEM RSA private key in a seed file.	Removed all A05 / API8 / API9 coverage in run -a	Fixed (1f2fa23)
4	Unclassified-surface shards admit non-target files	244 test/spec files and 10 vendored or minified assets seeded, including a single 853 KB third-party graphics library occupying its own shard.	No direct ground-truth loss; wastes budget that proportional sizing is meant to protect	Open
5	One ground-truth entry is unreachable by deliberate design	Its location sits in a file on the permanent seed denylist.	1 entry (retrieveBlueprintChallenge) can never be found	Accepted

7. Recording a future run

Append one JSON object per line to the appropriate file in `results/eval-history/`, then regenerate this PDF with `python3 tools/eval/generate_eval_report.py`. Every field in section 3.1 must be populated; use null where a value is genuinely unestablished rather than estimating it.

File	Holds
results/eval-history/scanner.jsonl	Runs of this harness's own scanner, any stage subset. Record which stages ran in the component field.
results/eval-history/external-baseline.jsonl	Third-party tool comparisons. Frozen unless a new external tool is benchmarked.
(future) results/eval-history/patcher.jsonl	Patch/fix phase, once that component exists.
(future) results/eval-history/verifier.jsonl	Verify phase, once that component exists.

Two rules that keep this log trustworthy: never overwrite a historical record, even when a run is later found to be invalid — annotate it in the notes field instead, as run -a is annotated for its failed lane. And never record a score without also recording the defects known to be in force at that time, or the number will be re-read later as a clean measurement of reasoning quality.